

What is Dependency Injection?

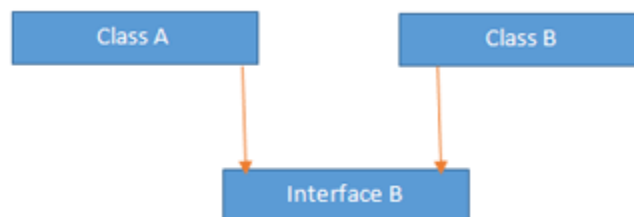
Dependency injection (DI) is a software design pattern that allows us to **separate the dependencies of a class from its implementation**. This makes **our code more loosely coupled and easier to test**.

Simple case to understand the DI using IOC(Inversion of Control)



- Here the Class A object uses the services provided by Class B
- The Class A object is depending on Class B.
- **But the best practices of designing says that two class objects should not be tightly coupled and so, we need to break the coupling between the two classes.**

An Interface is the mechanism for breaking this tight coupling between the two objects.



Here the Class A object knows what type of class object it is going to use, but it has information on what is the service or operations it is going to consume by Interface type. At run time, the program that is going to create an instance of Class A is going to pass the actual class object as a parameter to its constructor or the set method of the public property of the class.

This avoids the direct dependencies between the concrete classes. Here is the sample code that we are going to use:

```
using System;

// ===== Interfaces =====

public interface IClassA
{
    void DoSomethingFromClassB();
}

public interface IClassB
{
    void DoSomething();
}

// ===== Classes =====

public class ClassB : IClassB
{
    public void DoSomething()
    {
        Console.WriteLine("I am doing Something now");
    }
}

public class ClassA : IClassA
{
    public void DoSomethingFromClassB()
    {
```

```

        IClassB classB = new ClassB();
        classB.DoSomething();
    }
}

// ===== Program =====

class Program
{
    static void Main(string[] args)
    {
        IClassA classA = new ClassA();
        classA.DoSomethingFromClassB();
    }
}

```

Problem:

- Tight coupling
- Difficult to test
- Hard to replace **ClassB**

Further Improvement (Best Practice - Dependency Injection)

Instead of creating `ClassB` inside `ClassA`, inject it through the constructor.

using System;

```
public interface IClassA
{
    void DoSomethingFromClassB();
}
```

```
public interface IClassB
{
    void DoSomething();
}
```

```
public class ClassB : IClassB
{
    public void DoSomething()
    {
        Console.WriteLine("I am doing Something now");
    }
}
```

```
public class ClassA : IClassA
{
    private readonly IClassB _classB;

    public ClassA(IClassB classB)
    {
        _classB = classB;
    }
}
```

```
public void DoSomethingFromClassB()
{
    _classB.DoSomething();
}
}
```

```
class Program
{
    static void Main(string[] args)
    {
        IClassB classB = new ClassB();
        IClassA classA = new ClassA(classB);

        classA.DoSomethingFromClassB();
    }
}
```

This version follows the **Dependency Inversion Principle (DIP)** and is the preferred approach in real-world C# applications because **ClassA** depends on the **IClassB** abstraction rather than creating **ClassB** directly.

IoC and DI — Are they the same?

No, they are not exactly the same.

- IoC (Inversion of Control) is a design principle.
- DI (Dependency Injection) is a design pattern/technique used to achieve IoC.

Simple meaning

IoC:

A class does not create the objects it depends on. The objects are provided from outside.

DI:

The dependency is injected into the class, usually through the constructor.

Example

Without DI:

```
public class ClassA
{
    public void Show()
    {
        ClassB obj = new ClassB();
        obj.DoSomething();
    }
}
```

Here, **ClassA** creates **ClassB** itself → tight coupling.

With DI:

```
public class ClassA
{
    private readonly IClassB _classB;

    public ClassA(IClassB classB)
    {
        _classB = classB;
    }

    public void Show()
    {
        _classB.DoSomething();
    }
}
```

Here, **ClassB** is supplied from outside → Dependency Injection.

Easy way to remember

IoC = Principle

DI = Technique/Pattern to achieve IoC

There are 3 main types of Dependency Injection:

1. Constructor Injection

Dependency is passed through the constructor.

```
public class ClassA
{
    private readonly IClassB _classB;

    public ClassA(IClassB classB)
    {
        _classB = classB;
    }
}
```

Most commonly used in ASP.NET Core.

2. Property Injection

Dependency is passed through a property.

```
public class ClassA
{
    public IClassB ClassB { get; set; }

    public void Show()
    {
        ClassB.DoSomething();
    }
}
```

3. Method Injection

The dependency is passed through a method parameter.

```
public class ClassA
{
    public void Show(IClassB classB)
    {
        classB.DoSomething();
    }
}
```

Implementing Dependency Injection In .NET Core

In .NET Core, dependency injection is implemented using the `IServiceProvider` interface. This interface provides a way to register services with the DI container and then inject those services into classes that need them.

To use dependency injection in .NET Core, we need to do the following:

1. Create an interface that defines the contract for our dependency.
2. Create a class that implements the interface.
3. Register the service with the DI container.
4. Inject the service into the class that needs it.

- 1) Open Visual Studio Code and create a new project

Dotnet new webapi

- 2) let's create a **category model** with two fields - `CategoryId` and `CategoryName`.

```
namespace Dependency_InJection_Demo
{
    public class Category
    {
        public int CategoryId { get; set; }
        public string CategoryName { get; set; }
    }
}
```

- 3) Create an **interface of the repository** '`ICategoryRepository.cs`' in **Interfaces folder** having the `GetCategories` method, which returns the list of category objects.

```
using System.Collections.Generic;
```

```
namespace Dependency_InJection_Demo
{
    public interface ICategoryRepository
    {
        List<Category> GetCategories();
    }
}
```

4) Create **the Repositories folder add CategoryRepository.cs** Implement the preceding interface and return some sample data.

using System.Collections.Generic;

```
namespace Dependency_InJection_Demo
{
    public class CategoryRepository : ICategoryRepository
    {
        public List<Category> GetCategories()
        {
            List<Category> categories = new List<Category>();

            Category category = new Category() { CategoryId = 1, CategoryName =
"Category1" };
            categories.Add(category);

            category = new Category() { CategoryId = 2, CategoryName = "Category2" };
            categories.Add(category);

            return categories;
        }
    }
}
```

5) How will we expose the GET method from API?

We used to create an instance of CategoryRepository and call the GetCategories method using that instance. So tomorrow, if there is a change in CategoryRepository, it will directly affect the GET method of API as it is tightly coupled with that.

```
using Microsoft.AspNetCore.Mvc;
using System.Collections.Generic;
using System.Threading.Tasks;

namespace Dependency_Injection_Demo.Controllers
{
    [Route("api/Category")]

    public class CategoryController : Controller
    {
        private ICategoryRepository categoryRepository { get; set; }

        public CategoryController(ICategoryRepository categoryRepository)
        {
            this.categoryRepository = categoryRepository;
        }

        [HttpGet]
        public async Task<IActionResult> Get()
        {
            List<Category> categories = categoryRepository.GetCategories();
            return Ok(categories);
        }
    }
}
```

6) in .NET Core, Microsoft has provided an in-built container. We need to add the namespace, i.e., Microsoft.Extensions.DependencyInjection.

So, in the **Program.cs**, add service , we need to add our dependency into the service collection, which will dynamically inject whenever and wherever we want in the project.

- **Transient → Every time → New object**

Transient: It **creates an instance each time** they are requested and are never shared. It is used mainly for **lightweight stateless services**.

- **Singleton → Entire application → One object**

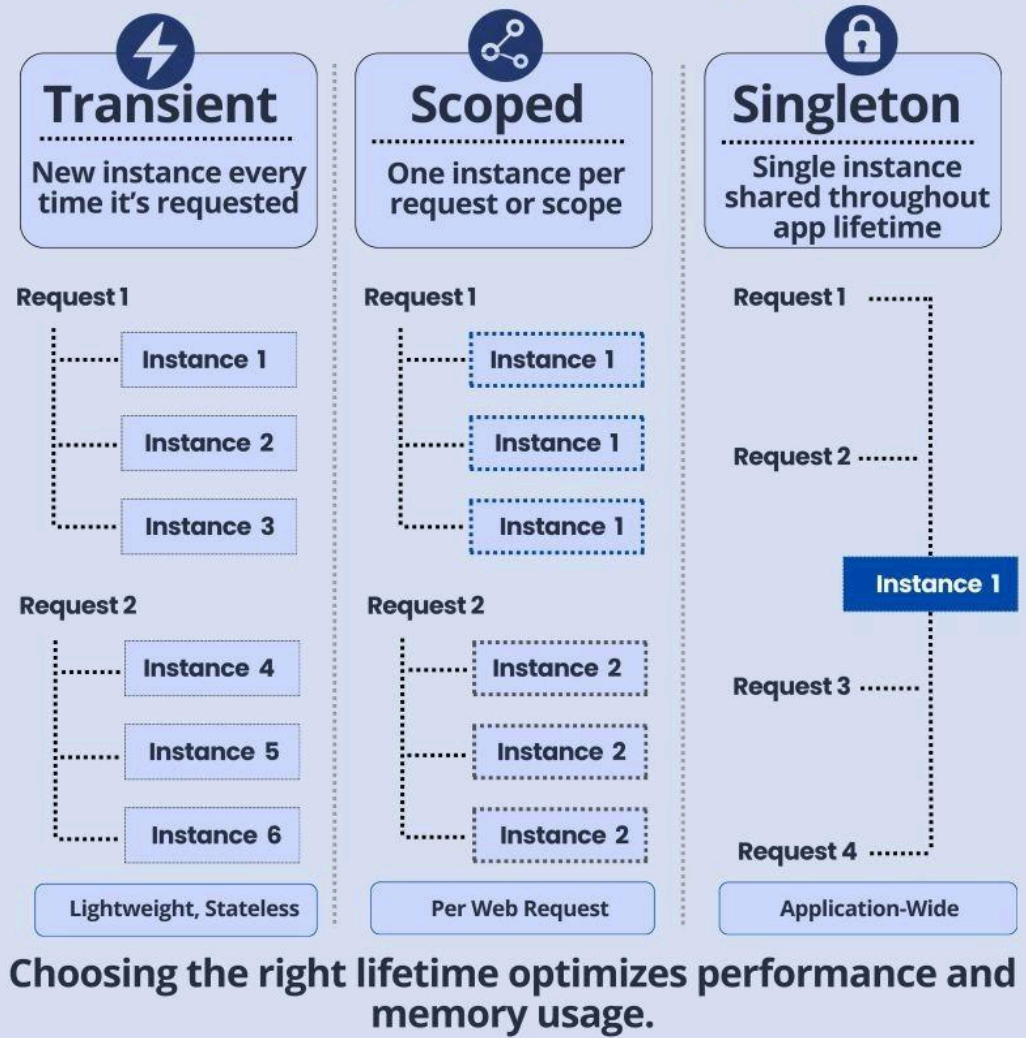
Singleton: This **creates only single instances** which are shared among all components that require it.

- **Scoped → Every request → One object**

Scoped:It creates an **instance once per scope, which is created on every request** to the Application.

ASP.Net Service Lifetimes Simplified

Understanding Dependency Injection



Lifetime	What to register	Example
Scoped	Repository / DB-related services	<code>StudentRepository</code> , <code>BookHelper</code>
Transient	Small, stateless services	<code>EmailService</code> , <code>CalculationService</code>
Singleton	Shared application-wide services	Configuration/cache-type services

//Program.cs

//Add Dependency Package

```
using Dependency_Injection_Demo;  
using Microsoft.Extensions.DependencyInjection;
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
// Add services to the container.
```

```
builder.Services.AddControllers();  
// Learn more about configuring Swagger/OpenAPI at  
https://aka.ms/aspnetcore/swashbuckle  
builder.Services.AddEndpointsApiExplorer();  
builder.Services.AddSwaggerGen();
```

//Add Interface & Repository using DI

// AddSingleton: One instance for the entire application

```
builder.Services.AddSingleton<ICategoryRepository, CategoryRepository>();
```

// AddTransient: New instance every time it's requested

```
// builder.Services.AddTransient<ICategoryRepository, CategoryRepository>();
```

// AddScoped: One instance per HTTP request

```
//builder.Services.AddScoped<ICategoryRepository, CategoryRepository>();
```

```
var app = builder.Build();
```

```
// Configure the HTTP request pipeline.  
if (app.Environment.IsDevelopment())  
{  
    app.UseSwagger();  
    app.UseSwaggerUI();  
}
```

```
app.UseHttpsRedirection();
```

```
app.UseAuthorization();
```

```
app.MapControllers();
```

```
app.Run();
```

7) So far, we have added our dependency to the collection.

It's time to inject where we need it, i.e., in the Web API. Our GET method is inside the `CategoryController`, and we want an instance of `categoryrepository`. So, let's create a `CategoryController` constructor which expects the `ICategoryRepository` type.

From this parameterized constructor, set the private property of type `ICategoryRepository`, which will be used to call `GetCategories` from the GET method.

```
using Microsoft.AspNetCore.Mvc;  
using System.Collections.Generic;  
using System.Threading.Tasks;
```

```
namespace DlinCore.Controllers  
{  
    [Route("api/Category")]  
    public class CategoryController : Controller  
    {  
        private ICategoryRepository categoryRepository { get; set; }  
  
        public CategoryController(ICategoryRepository categoryRepository)  
        {  
            this.categoryRepository = categoryRepository;  
        }  
  
        [HttpGet]  
        public async Task<ActionResult> Get()
```

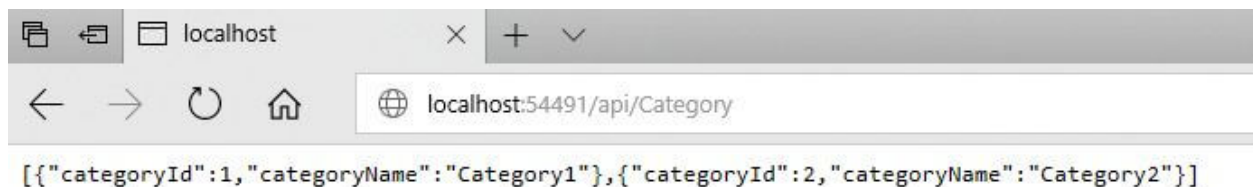
```

    {
        List<Category> categories = categoryRepository.GetCategories();
        return Ok(categories);
    }
}

```

8) Run the application,

we will be able to see the result of the GET method of CategoryController. Now, even though we haven't created an instance of CategoryRepository, which is expected by CategoryController, we are able to call the GET method successfully. The instance of CategoryRepository has been resolved dynamically, i.e., our Dependency Injection.



Advantages of using dependency injection in .NET Core

Here are some of the advantages of using dependency injection in .NET Core:

- **Loose coupling:** Dependency injection allows us to decouple our classes from their dependencies. This makes our code more maintainable and easier to test.
- **Testability:** Dependency injection makes our code more testable because we can mock out dependencies in our unit tests.
- **Extensibility:** Dependency injection makes our code more extensible because we can easily swap out dependencies.
- **Reusability:** Dependency injection makes our code more reusable because we can easily share dependencies between different classes.

What are the different types of dependency injection?

There are two main types of dependency injection: constructor injection and property injection.

- **Constructor injection:** Constructor injection is the most common type of dependency injection. In constructor injection, we inject the dependency into the class constructor.
- **Property injection:** Property injection is less common than constructor injection. In property injection, we inject the dependency into a property of the class.

